

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

Task -1: Create Project Structure as follows:

```
OSSP/
|
|— .git/           # Git repository (created automatically)
|— .gitignore      # Files/folders Git should ignore
|— README.md       # Project description and instructions
|— LICENSE         # Project license (optional)
|— Makefile        # Build instructions
|
|— src/            # Source code
|   |— main.c
|   |— shell.c
|   |— parser.c
|   |— executor.c
|   |— builtin.c
|
|— include/        # Header files
|   |— shell.h
|   |— parser.h
|   |— executor.h
|   |— builtin.h
|
|— obj/            # Object files (.o) (generated)
|
|— bin/            # Executable file (generated)
|   |— my_shell
|
|— docs/           # Documentation
|   |— design.md
|   |— report.pdf
|
|— tests/          # Test programs
|   |— test_parser.c
|   |— test_executor.c
```

```
|
|├── scripts/          # Helper scripts
|   └── run.sh
|
|└── assets/          # Images/screenshots
└── architecture.png
```

Note: For creating this structure use mkdir and touch commands.

```
navaneeth@Navaneeth: ~/OS  ×  +  ∨  
navaneeth@Navaneeth:~$ cd ~/OSSP  
navaneeth@Navaneeth:~/OSSP$ tree -a -L 2
```

```
.  
├── .git  
│   ├── COMMIT_EDITMSG  
│   ├── HEAD  
│   ├── config  
│   ├── description  
│   ├── hooks  
│   ├── index  
│   ├── info  
│   ├── logs  
│   ├── objects  
│   └── refs  
├── .gitignore  
├── LICENSE  
├── Makefile  
├── README.md  
├── assets  
│   └── architecture.png  
├── bin  
│   └── my_shell  
├── docs  
│   ├── design.md  
│   └── report.pdf  
├── fork_exec  
├── fork_exec.c  
├── include  
│   ├── builtin.h  
│   ├── executor.h  
│   ├── parser.h  
│   └── shell.h  
├── obj  
├── scripts  
│   └── run.sh  
├── src  
│   ├── builtin.c  
│   ├── executor.c  
│   ├── main.c  
│   ├── parser.c  
│   └── shell.c  
└── tests  
    ├── test_executor.c  
    └── test_parser.c
```

```
15 directories, 27 files
```

```
navaneeth@Navaneeth:~/OSSP$ |
```

2. To Analyze process abstraction, execute fork(), understand exec() family, analyze parentchild relationships, inspect process tree, practice system call tracing.

Task-1: Using a manual command understand the fork() and exec() system calls

Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

Example:

```
#include <stdio.h>
#include <unistd.h>
int main() {    pid_t
pid;
    pid = fork(); // Create a new process
if (pid < 0) {
    printf("Fork failed!\n");
}
    else if (pid == 0) {
// Child Process
        printf("This is the Child Process.\n");
printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
printf("Parent PID = %d\n", getpid());    printf("Child
PID = %d\n", pid);
    }
return 0;
}
```

navaneeth@Navaneeth: ~/OS × + ▾

GNU nano 8.7.1

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
    }
    else if (pid == 0)
    {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

```
navaneeth@Navaneeth:~/OSSP$ nano fork_exec.c
navaneeth@Navaneeth:~/OSSP$ gcc fork_exec.c -o fork_exec
navaneeth@Navaneeth:~/OSSP$ ./fork_exec
This is the Parent Process.
Parent PID = 507
Child PID = 508
This is the Child Process.
Child PID = 508
navaneeth@Navaneeth:~/OSSP$ |
```